

Algorithmes PCC

Principe de l'algorithme de Dijkstra.

- Maintient un ensemble $FERME$ de sommets pour lesquels l'estimation $d(x) = S(x_0, x) \leftarrow$
- sélectionne un sommet y de $OUVERT$ dont l'estimation $d(y)$ est la plus faible.
- L'insère dans $FERME$
- Relâche tous les arcs sortant de y .

Deux autres écritures de l'algorithme de Dijkstra.

Dijkstra ($G = (S, A, w)$, $x_0 \in S$)

{ Initialisation-Proc ($G = (S, A, w)$, $x_0 \in S$).

FERME $\leftarrow \emptyset$

$\mathcal{Q} \leftarrow S$

Tant que $\mathcal{Q} \neq \emptyset$ faire {

$x \leftarrow \text{Extract_Minimum}(\mathcal{Q})$;

Pour chaque sommet $y \in \text{succ}(x)$ faire {

Relaxation ($x, y, c(x, y)$) }

FERME $\leftarrow \text{FERME} \cup \{x\}$ }

Retourner π, d .

⚠ Dans l'algorithme de Dijkstra un sommet est traité une et une seule fois, d'où une procédure de marquage est nécessaire pour marquer les sommets

Dijkstra $(G = (S, A, w), x_0 \in S)$ }

% Initialisation.

Pour chaque sommet $x \in S$ faire {

$d[x] \leftarrow +\infty$; $\pi[x] \leftarrow \text{null}$; couleur[x] $\leftarrow b$ }

$d[x_0] \leftarrow 0$; couleur[x₀] $\leftarrow g$; OUVERT $\leftarrow \{x_0\}$.

% Boucle Principale.

Tant que OUVERT $\neq \emptyset$

faire {

$x \leftarrow \text{ExtraireMinimum}(\text{OUVERT})$; couleur[x] $\leftarrow n$;

Pour chaque $y \in \text{succ}(x)$ faire {

si couleur[y] $\neq n$ alors {

si $d[y] > d[x] + c(x, y)$ alors {

$d[y] \leftarrow d[x] + c(x, y)$

$\pi[y] \leftarrow x$;

si couleur[y] $= b$ alors { couleur[y] $\leftarrow g$;

$\text{OVERT} \Leftarrow \text{OVERT} \cup y \downarrow$
 $\downarrow \downarrow$

return $\pi, \downarrow$

Algorithme de Dijkstra.

Attention : valide uniquement si les valuations sont ≥ 0

```
 $\lambda[0] \leftarrow 0$ ; visité[0]  $\leftarrow$  vrai;  $P[0] \leftarrow 0$ ;  
pour  $i \leftarrow 1$  à  $n-1$  faire {  $\lambda[i] \leftarrow \infty$ ; visité[i]  $\leftarrow$  faux;  $P[i] \leftarrow -1$ ; }  
pour chaque successeurs  $j$  de 0 faire  $\lambda[j] \leftarrow v_{0,j}$ ;  
nb_visités  $\leftarrow 1$ ;  
tant que (nb_visités  $< n$ ) {  
    min  $\leftarrow -1$ ;  
    pour  $i \leftarrow 1$  à  $n-1$  faire {  
        si (visité[i]=faux et (min=-1 ou  $\lambda[i] < \lambda[\text{min}]$ ) ) alors min  $\leftarrow i$ ;  
    }  
    visité[min]  $\leftarrow$  vrai; nb_visités  $\leftarrow$  nb_visités + 1;  
    pour chaque successeurs  $j$  de min faire {  
        si (visité[j]=faux et  $\lambda[j] > \lambda[\text{min}] + v_{\text{min},j}$ ) alors {  
             $\lambda[j] \leftarrow \lambda[\text{min}] + v_{\text{min},j}$ ;  
             $P[j] \leftarrow \text{min}$ ;  
        }  
    }  
}
```

→ Algorithme de Dijkstra Modifié

```
 $\lambda[0] \leftarrow 0$ ; visité[0]  $\leftarrow$  vrai;  $P[0] \leftarrow 0$ ;  
pour  $i \leftarrow 1$  à  $n-1$  faire {  $\lambda[i] \leftarrow \infty$ ; visité[i]  $\leftarrow$  faux;  $P[i] \leftarrow -1$ ; }  
pour chaque successeurs  $j$  de 0 faire  $\lambda[j] \leftarrow v_{0,j}$ ;  
nb_visités  $\leftarrow 1$ ;  
tant que (nb_visités  $< n$ ) {  
  (min,  $v_{\min}$ )  $\leftarrow$  extraire_min_tas(TAS);  
   $\lambda[\text{min}] \leftarrow v_{\min}$ ;  
  nb_visités  $\leftarrow$  nb_visités + 1;  
  pour chaque successeurs  $j$  de min faire {  
    si ( $\lambda[j] > \lambda[\text{min}] + v_{\text{min},j}$ ) alors {  
       $\lambda[j] \leftarrow \lambda[\text{min}] + v_{\text{min},j}$ ;  
       $P[j] \leftarrow \text{min}$ ;  
      rétablir_tas(TAS, j);  
    }  
  }  
}
```

$x_0, x_1, \dots, x_{n-1}$

$\lambda : S \rightarrow \mathbb{R}$

$x_i \rightarrow \lambda[x_i] = \lambda[i]$

visité : $S \rightarrow \{0, 1\}$

$x_0 \mapsto \text{visité}[x_i] = \text{visité}[i]$

$P : S \rightarrow S$

$x_i \mapsto P[x_i] = P[i]$

→ Programmation dynamique pour le calcul de plus courts chemins.

↳ Principe des approches "Diviser pour régner":

- Décomposer le problème à résoudre en un ensemble de sous-problème.
- Résoudre chaque sous-problème
- calculer la solution du problème initial à partir des solutions des sous-problèmes.

Exemples: Quicksort, Merge Sort, Branch-and-Bound

Reue: Approche "Diviser pour régner" où ^Aun même sous-problème peut apparaître plusieurs fois ☐
(Programmation dynamique)

↳ Étapes pour résoudre un problème en programmation dynamique.

Étape 1 ; Définir ce qu'est un sous-problème.

Étape 2 : Définir récursivement la solution d'un sous-problème

- Identifier les cas de base, où la solution est connue sans calcul.
- Définir la solution aux sous-problèmes plus compliqués en fonction des solutions de sous-problèmes plus simples.

Eq-Bellman basée sur le principe d'optimalité des sous-structures.

Étape 3 : Analyse des performances.

Combien de sous-problèmes différents doivent être résolus.

Étape 4 : choisir une approche pour éviter de recalculer les solutions (résoudre un sous-problème plus qu'une fois).

- Top-down : Implémentation récursive + mémorisation.

- Bottom-down : Implémentation itérative, en remplissant un tableau.

⇒ parfois plus économique en mémoire.